



DWB层实现

DWB基础数据层，对经常使用的明细数据进行降维冗余形成宽表，一般用作中间层。

将当前有效的基础数据关联查询出来，覆盖保存。为DWS和DM的聚合统计作准备。

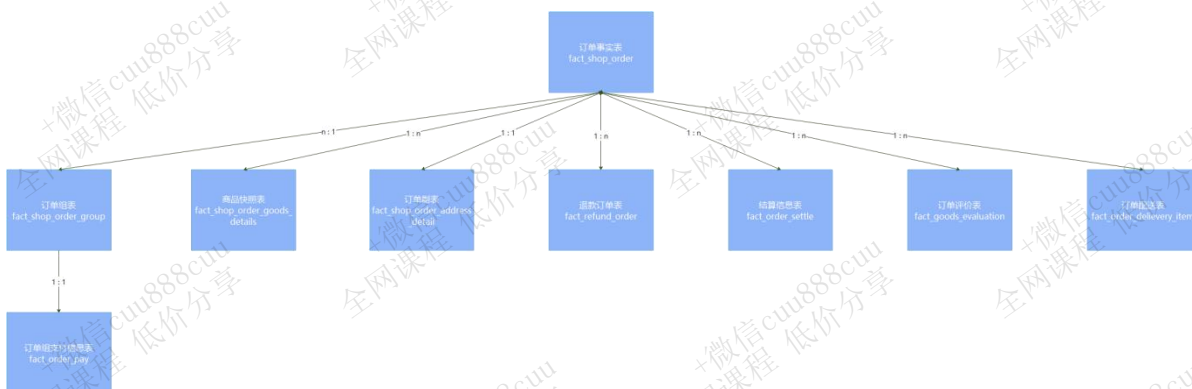
注：宽表从字面意义上讲就是字段比较多的数据库表。通常是指业务主题相关的指标、维度、属性关联在一起的一张数据库表。由于把不同的内容都放在同一张表存储，宽表已经不符合三范式的模型设计规范，随之带来的主要坏处就是数据的大量冗余，与之相对应的好处就是查询性能的提高与便捷。宽表通过把相关字段放在同一张表中，减少join操作，可以大大提高数据计算时的效率问题。

后期随着统计需求的迭代，DWB冗余的表和字段也会发生变化。

1. 建库

```
create database yp_dwb;
```

2. 订单明细宽表





2.1 建表

```
DROP TABLE if EXISTS yp_dwb.dwb_order_detail;
CREATE TABLE yp_dwb.dwb_order_detail(
    order_id string COMMENT '根据一定规则生成的订单编号',
    order_num string COMMENT '订单序号',
    buyer_id string COMMENT '买家的userId',
    store_id string COMMENT '店铺id',
    order_from string COMMENT '渠道类型: android、ios、miniapp、pweb、other',
    order_state int COMMENT '订单状态:1.已下单\; 2.已付款, 3. 已确认 \;4.配送\; 5.已完成\; 6.退款\;7.已取消',
    create_date string COMMENT '下单时间',
    finnshed_time timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动评价',
    is_settlement tinyint COMMENT '是否结算\;0.待结算订单\; 1.已结算订单\;',
    is_delete tinyint COMMENT '订单评价的状态:0.未删除\; 1.已删除\;(默认0)',
    evaluation_state tinyint COMMENT '订单评价的状态:0.未评价\; 1.已评价\;(默认0)',
    way string COMMENT '取货方式:SELF自提\;SHOP店铺负责配送',
    is_stock_up int COMMENT '是否需要备货 0: 不需要    1: 需要    2:平台确认备货 3:已完成备货 4平台已经将货物送至店铺 ',
    -- 订单副表
    order_amount decimal(36,2) COMMENT '订单总金额:购买总金额-优惠金额',
    discount_amount decimal(36,2) COMMENT '优惠金额',
    goods_amount decimal(36,2) COMMENT '用户购买的商品的总金额+运费',
    is_delivery string COMMENT '0.自提; 1.配送',
    buyer_notes string COMMENT '买家备注留言',
    pay_time string,
    receive_time string,
    delivery_begin_time string,
    arrive_store_time string,
    arrive_time string COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动评价',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid tinyint COMMENT '是否有效 0: false\; 1: true\; 订单是否有效的标志',
    -- 订单组
    group_id string COMMENT '订单分组id',
    is_pay tinyint COMMENT '订单组是否已支付,0未支付,1已支付',
    -- 订单组支付
    group_pay_amount decimal(36,2) COMMENT '订单总金额\;',
    -- 退款单
    refund_id string COMMENT '退款单号',
    apply_date string COMMENT '用户申请退款的时间',
    refund_reason string COMMENT '买家退款原因',
    refund_amount decimal(36,2) COMMENT '订单退款的金额',
    refund_state tinyint COMMENT '1.申请退款\;2.拒绝退款\; 3.同意退款,配送员配送\; 4:商家同意退款,用户亲自送货 \;5.退款完成',
    -- 结算单
```





```

settle_id string COMMENT '结算单号',
settlement_amount decimal(36,2) COMMENT '如果发生退款,则结算的金额 = 订单的总金额 - 退款的金额',
dispatcher_user_id string COMMENT '配送员id',
dispatcher_money decimal(36,2) COMMENT '配送员的配送费(配送员的运费(如果退货方式为1:则买家支付配送费))',
circle_master_user_id string COMMENT '圈主id',
circle_master_money decimal(36,2) COMMENT '圈主分润的金额',
plat_fee decimal(36,2) COMMENT '平台应得的分润',
store_money decimal(36,2) COMMENT '商家应得的订单金额',
status tinyint COMMENT '0.待结算; 1.待审核 \; 2.完成结算; 3.拒绝结算',
settle_time string COMMENT ' 结算时间',
-- 订单评价
evaluation_id string,
evaluation_user_id string COMMENT '评论人id',
geval_scores int COMMENT '综合评分',
geval_scores_speed int COMMENT '送货速度评分0-5分(配送评分)',
geval_scores_service int COMMENT '服务评分0-5分',
geval_isanony tinyint COMMENT '0-匿名评价, 1-非匿名',
evaluation_time string,
-- 订单配送
delievery_id string COMMENT '主键id',
dispatcher_order_state tinyint COMMENT '配送订单状态:0.待接单.1.已接单,2.已到店.3.配送中 4.商家普通提货码完成订单.5.商家万能提货码完成订单.6. 买家完成订单',
delivery_fee decimal(36,2) COMMENT '配送员的运费',
distance int COMMENT '配送距离',
dispatcher_code string COMMENT '收货码',
receiver_name string COMMENT '收货人姓名',
receiver_phone string COMMENT '收货人电话',
sender_name string COMMENT '发货人姓名',
sender_phone string COMMENT '发货人电话',
delievery_create_time string,
-- 商品快照
order_goods_id string COMMENT '--商品快照id',
goods_id string COMMENT '购买商品的id',
buy_num int COMMENT '购买商品的数量',
goods_price decimal(36,2) COMMENT '购买商品的价格',
total_price decimal(36,2) COMMENT '购买商品的价格 = 商品的数量 * 商品的单价 ',
goods_name string COMMENT '商品的名称',
goods_specification string COMMENT '商品规格',
goods_type string COMMENT '商品分类      ytgj:进口商品      ytsc:普通商品      hots爆品',
goods_brokerage decimal(36,2) COMMENT '商家设置的商品分润的金额',
is_goods_refund tinyint COMMENT '0.不退款\; 1.退款'
)
COMMENT '订单明细表'
PARTITIONED BY(dt STRING)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');

```





2.2 导入数据

问题：多表关联时，where过滤要放在子查询中，以确保优先执行吗？

2.2.1 首次

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
--分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;

insert into yp_dwb.dwb_order_detail partition (dt)
select
    o.id as order_id,
    o.order_num,
    o.buyer_id,
    o.store_id,
    o.order_from,
    o.order_state,
    o.create_date,
    o.finnshed_time,
    o.is_settlement,
    o.is_delete,
    o.evaluation_state,
    o.way,
    o.is_stock_up,
    od.order_amount,
    od.discount_amount,
    od.goods_amount,
    od.is_delivery,
```





```
od.buyer_notes,  
od.pay_time,  
od.receive_time,  
od.delivery_begin_time,  
od.arrive_store_time,  
od.arrive_time,  
od.create_user,  
od.create_time,  
od.update_user,  
od.update_time,  
od.is_valid,  
og.group_id,  
og.is_pay,  
op.order_pay_amount as group_pay_amount,  
refund.id as refund_id,  
refund.apply_date,  
refund.refund_reason,  
refund.refund_amount,  
refund.refund_state,  
os.id as settle_id,  
os.settlement_amount,  
os.dispatcher_user_id,  
os.dispatcher_money,  
os.circle_master_user_id,  
os.circle_master_money,  
os.plat_fee,  
os.store_money,  
os.status,  
os.settle_time,  
  
e.id,  
e.user_id,  
e.geval_scores,  
e.geval_scores_speed,  
e.geval_scores_service,  
e.geval_isanony,  
e.create_time,  
d.id,  
d.dispatcher_order_state,  
d.delivery_fee,  
d.distance,  
d.dispatcher_code,
```





```
d.receiver_name,
d.receiver_phone,
d.sender_name,
d.sender_phone,
d.create_time,

ogoods.id as order_goods_id,
ogoods.goods_id,
ogoods.buy_num,
ogoods.goods_price,
ogoods.total_price,
ogoods.goods_name,
ogoods.goods_specification,
ogoods.goods_type,
ogoods.goods_brokerage,
ogoods.is_refund as is_goods_refund,
SUBSTRING(o.create_date,1,10) as dt
--订单表
from yp_dwd.fact_shop_order o
--订单副表
left join yp_dwd.fact_shop_order_address_detail od on o.id = od.id and od.end_date='9999-99-99'
--订单组表
left join yp_dwd.fact_shop_order_group og on o.id = og.order_id and og.end_date='9999-99-99'
--订单组支付表
left join yp_dwd.fact_order_pay op on og.group_id = op.group_id
--退款单表
left join yp_dwd.fact_refund_order refund on refund.order_id=o.id and
refund.end_date='9999-99-99'
--结算单表
left join yp_dwd.fact_order_settle os on os.order_id=o.id and os.end_date='9999-99-99'
--订单商品快照
left join yp_dwd.fact_shop_order_goods_details ogoods on ogoods.order_id=o.id and
ogoods.end_date='9999-99-99'
--订单评价表
left join yp_dwd.fact_goods_evaluation e on e.order_id=o.id and e.is_valid=1
--订单配送表
left join yp_dwd.fact_order_delievery_item d on d.shop_order_id=o.id and
d.dispatcher_order_type=1 and d.is_valid=1 and d.end_date='9999-99-99'
where o.end_date='9999-99-99'
--and og.group_id='dd2019022817332967452f41f'
--and o.id='dd19040334655236d5'
```



2.2.2 循环

订单流程有固定的时间限制：下单后有支付时间限制、支付后的配送流程有时间限制、收货后有评价时间限制(超时自动好评)。

确定业务更新周期后，只需要将更新周期内的数据覆盖插入即可。通过`create_date`限制数据范围，通过`start_date`走分区提高性能。

```
--覆盖插入
insert overwrite table yp_dwb.dwb_order_detail partition (dt)
select
    .....,
    SUBSTRING(o.create_date,1,10) as dt
--订单表
from yp_dwd.fact_shop_order o
--订单副表
left join yp_dwd.fact_shop_order_address_detail od on o.id = od.id and od.end_date='9999-99-99'
--订单组表--订单组支付表--退款单表--结算单表--订单商品快照--订单评价表--订单配送表
left join .....
where o.end_date='9999-99-99'
-- 读取上个月1日至今的数据
and SUBSTRING(o.create_date,1,10) >= '${Last_Month_DATE}' and o.start_date >=
`${Last_Month_DATE}`
;
```

2.3 MapJoin

2.3.1 Map Join

MapJoin顾名思义，就是在Map阶段进行表之间的连接。而不需要进入到Reduce阶段才进行连接。这样就节省了在Shuffle阶段时要进行的大量数据传输。从而起到了优化作业的作用。

要使MapJoin能够顺利进行，那就必须满足这样的条件：除了一份表的数据分布在不同的Map中外，其他连接的表的数据必须在每个Map中有完整的拷贝。



所以并不是所有的场景都适合用MapJoin。它通常会用在如下的一些情景：在二个要连接的表中，有一个很大，有一个很小，这个小表可以存放在内存中而不影响性能。

这样我们就把小表文件复制到每一个Map任务的本地，再让Map把文件读到内存中待用。

在Hive v0.7之前，需要使用hint提示 /*+ mapjoin(table) */才会执行MapJoin。Hive v0.7之后的版本已经不需要给出MapJoin的指示就进行优化。现在可以通过如下配置参数来进行控制：

```
set hive.auto.convert.join=true;
```

Hive还提供另外一个参数--表文件的大小作为开启和关闭MapJoin的**阈值**：

```
--旧版本为hive.mapjoin.smalltable.filesize
```

```
set hive.auto.convert.join.noconditionaltask.size=512000000
```

注意，如果hive.auto.convert.join是关闭的，则本参数不起作用。否则，如果参与连接的N个表(或分区)中的N-1个 的总大小小于512MB，则直接将连接转为Map连接。默认值为20MB。

启用 MapJoin 优化

hive.auto.convert.join

Hive 自动转换联接无条件大小

hive.auto.convert.join.noconditionaltask.size

☒ HiveServer2 Default Group

HiveServer2 Default Group

100

兆字节

MapJoin的使用场景：

1. 关联操作中有一张表非常小
2. 不等值的链接操作

2.3.1.1 大小表关联

```
select f.a,f.b from A t join B f on (f.a=t.a and f.ftime=20110802)
```

该语句中B表有30亿行记录，A表只有100行记录，而且B表中数据倾斜特别严重，有一个key上有15亿行记录，在运行过程中特别的慢，而且在reduce的过程中遇到执行时间过长或者内存不够的问题。

MAPJOIN会把小表全部读入内存中，在map阶段直接拿另外一个表的数据和内存中表数据做匹配，由于在map时进行了join操作，省去了reduce运行的效率会高很多。

这样就不会由于数据倾斜导致某个reduce上落数据太多而失败。于是原来的sql可以通过使用



hint的方式指定join时使用mapjoin。

```
select /*+ mapjoin(A)*/ f.a,f.b from A t join B f on ( f.a=t.a and f.ftime=20110802)
```

在实际使用中，只要根据业务调整小表的阈值即可，hive会自动帮我们完成mapjoin，提高执行的效率。

2.3.1.2 不等连接

mapjoin还有一个很大的好处是能够进行不等连接的join操作，如果将不等条件写在where中，那么mapreduce过程中会进行笛卡尔积，运行效率特别低，如果使用mapjoin操作，在map的过程中就完成了不等值的join操作，效率会高很多。

```
select A.a ,A.b from A join B where A.a>B.a
```

2.3.2 Bucket-MapJoin

2.3.2.1 作用

两个表join的时候，**小表不足以放到内存中**，但是又想用map side join这个时候就要用到bucket Map join。其方法是两个join表在join key上都做**hash bucket**，并且把你打算复制的那个（相对）**小表的bucket数设置为大表的倍数**。这样数据就会按照key join，做hash bucket。小表依然复制到所有节点，Map join的时候，小表的每一组bucket加载成hashtable，与对应的一个大表bucket做**局部join**，这样每次只需要加载部分hashtable就可以了。

2.3.2.2 条件

- 1) set hive.optimize.bucketmapjoin = true;
- 2) 一个表的bucket数是另一个表bucket数的整数倍
- 3) bucket列 == join列
- 4) 必须是应用在map join的场景中

注意：如果表不是bucket的，则只是做普通join。

2.3.3 SMB Join



全称Sort Merge Bucket Join。

2.3.3.1 作用

大表对小表应该使用MapJoin来进行优化，但是如果是**大表对大表**，如果进行shuffle，那就非常可怕，第一个慢不用说，第二个容易出异常，此时就可以使用SMB Join来提高性能。SMB Join基于bucket-mapjoin的**有序bucket**，可实现在map端完成join操作，可以有效地减少或避免shuffle的数据量。SMB join的条件和Map join类似但又不同。

2.3.3.2 条件

bucket mapjoin	SMB join
set hive.optimize.bucketmapjoin = true;	set hive.optimize.bucketmapjoin = true; set hive.auto.convert.sortmerge.join=true; set hive.optimize.bucketmapjoin.sortedmerge = true; set hive.auto.convert.sortmerge.join.noconditionaltask=true;
一个表的bucket数是另一个表bucket数的整数倍	小表的bucket数=大表bucket数
bucket列 == join列	Bucket 列 == Join 列 == sort 列
必须是应用在map join的场景中	必须是应用在bucket mapjoin 的场景中

2.3.3.3 确保分桶列排序

hive并不检查两个join的表是否已经做好bucket且sorted，需要用户自己去**保证join的表数据sorted**，否则可能数据不正确。

有两个办法：

- 1) hive.enforce.sorting 设置为 true。开启强制排序时，插数据到表中会进行强制排序，默认false。
- 2) 插入数据时通过在sql中用distributed c1 sort by c1 或者 cluster by c1

另外，**表创建时必须是CLUSTERED且SORTED**，如下：





```
create table test_smb_2(mid string,age_id string)
CLUSTERED BY(mid) SORTED BY(mid) INTO 500 BUCKETS;
```

综上，涉及到分桶表操作的齐全配置为：

```
--写入数据强制分桶
set hive.enforce.bucketing=true;
--写入数据强制排序
set hive.enforce.sorting=true;
--开启bucketmapjoin
set hive.optimize.bucketmapjoin = true;
--开启SMB Join
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;
```

开启MapJoin的配置（hive.auto.convert.join和hive.auto.convert.join.noconditionaltask.size），还有限制对桶表进行load操作（hive.strict.checks.bucketing）可以直接设置在hive的配置项中，无需在sql中声明。

自动尝试SMB联接（hive.optimize.bucketmapjoin.sortedmerge）也可以在设置中进行提前配置。

2.3.4 验证

2.3.4.1 打开mapjoin

```
--开启mapjoin
set hive.auto.convert.join=true;
--写入数据强制分桶
set hive.enforce.bucketing=true;
--写入数据强制排序
set hive.enforce.sorting=true;
--开启bucketmapjoin
set hive.optimize.bucketmapjoin = true;
--开启SMB Join
set hive.optimize.bucketmapjoin.sortedmerge=true;
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;
```





```
-- insert into yp_dwb.dwb_order_detail partition (dt)
```

```
explain
```

```
select
```

```
o.id as order_id,  
o.order_num,  
o.buyer_id,  
o.store_id,  
o.order_from,  
o.order_state,  
o.create_date,  
o.finnshed_time,  
o.is_settlement,  
o.is_delete,  
o.evaluation_state,  
o.way,  
o.is_stock_up,  
od.order_amount,  
od.discount_amount,  
od.goods_amount,  
od.is_delivery,  
od.buyer_notes,  
od.pay_time,  
od.receive_time,  
od.delivery_begin_time,  
od.arrive_store_time,  
od.arrive_time,  
od.create_user,  
od.create_time,  
od.update_user,  
od.update_time,  
od.is_valid,  
og.group_id,  
og.is_pay,  
op.order_pay_amount as group_pay_amount,  
refund.id as refund_id,  
refund.apply_date,  
refund.refund_reason,  
refund.refund_amount,  
refund.refund_state,  
os.id as settle_id,  
os.settlement_amount,  
os.dispatcher_user_id,
```





```
os.dispatcher_money,
os.circle_master_user_id,
os.circle_master_money,
os.plat_fee,
os.store_money,
os.status,
os.settle_time,
e.id,
e.user_id,
e.geval_scores,
e.geval_scores_speed,
e.geval_scores_service,
e.geval_isanony,
e.create_time,
d.id,
d.dispatcher_order_state,
d.delivery_fee,
d.distance,
d.dispatcher_code,
d.receiver_name,
d.receiver_phone,
d.sender_name,
d.sender_phone,
d.create_time,

ogoods.id as order_goods_id,
ogoods.goods_id,
ogoods.buy_num,
ogoods.goods_price,
ogoods.total_price,
ogoods.goods_name,
ogoods.goods_specification,
ogoods.goods_type,
ogoods.goods_brokerage,
ogoods.is_refund as is_goods_refund,
SUBSTRING(o.create_date,1,10) as dt
--订单表
from yp_dwd.fact_shop_order o
--订单副表
left join yp_dwd.fact_shop_order_address_detail od on o.id = od.id and od.end_date='9999-99-99'
--订单组表
```



```
left join yp_dwd.fact_shop_order_group og on o.id = og.order_id and og.end_date='9999-99-99'

--订单组支付表
left join yp_dwd.fact_order_pay op on og.group_id = op.group_id

--退款单表
left join yp_dwd.fact_refund_order refund on refund.order_id=o.id and
refund.end_date='9999-99-99'

--结算单表
left join yp_dwd.fact_order_settle os on os.order_id=o.id and os.end_date='9999-99-99'

--订单商品快照
left join yp_dwd.fact_shop_order_goods_details ogoods on ogoods.order_id=o.id and
ogoods.end_date='9999-99-99'

--订单评价表
left join yp_dwd.fact_goods_evaluation e on e.order_id=o.id and e.is_valid=1

--订单配送表
left join yp_dwd.fact_order_delievery_item d on d.shop_order_id=o.id and
d.dispatcher_order_type=1 and d.is_valid=1 and d.end_date='9999-99-99'
where o.end_date='9999-99-99';
```

2.3.4.2 关闭mapjoin

```
--开启mapjoin
set hive.auto.convert.join=false;

--写入数据强制分桶
set hive.enforce.bucketing=false;

--写入数据强制排序
set hive.enforce.sorting=false;

--开启bucketmapjoin
set hive.optimize.bucketmapjoin = false;

--开启SMB Join
set hive.optimize.bucketmapjoin.sortedmerge=false;
set hive.auto.convert.sortmerge.join=false;
set hive.auto.convert.sortmerge.join.noconditionaltask=false;

-- insert into yp_dwb.dwb_order_detail partition (dt)
explain
select
    o.id as order_id,
    o.order_num,
    o.buyer_id,
    o.store_id,
```





```
o.order_from,
o.order_state,
o.create_date,
o.finnshed_time,
o.is_settlement,
o.is_delete,
o.evaluation_state,
o.way,
o.is_stock_up,
od.order_amount,
od.discount_amount,
od.goods_amount,
od.is_delivery,
od.buyer_notes,
od.pay_time,
od.receive_time,
od.delivery_begin_time,
od.arrive_store_time,
od.arrive_time,
od.create_user,
od.create_time,
od.update_user,
od.update_time,
od.is_valid,
og.group_id,
og.is_pay,
op.order_pay_amount as group_pay_amount,
refund.id as refund_id,
refund.apply_date,
refund.refund_reason,
refund.refund_amount,
refund.refund_state,
os.id as settle_id,
os.settlement_amount,
os.dispatcher_user_id,
os.dispatcher_money,
os.circle_master_user_id,
os.circle_master_money,
os.plat_fee,
os.store_money,
os.status,
os.settle_time,
```





```
e.id,  
e.user_id,  
e.geval_scores,  
e.geval_scores_speed,  
e.geval_scores_service,  
e.geval_isanony,  
e.create_time,  
d.id,  
d.dispatcher_order_state,  
d.delivery_fee,  
d.distance,  
d.dispatcher_code,  
d.receiver_name,  
d.receiver_phone,  
d.sender_name,  
d.sender_phone,  
d.create_time,  
  
ogoods.id as order_goods_id,  
ogoods.goods_id,  
ogoods.buy_num,  
ogoods.goods_price,  
ogoods.total_price,  
ogoods.goods_name,  
ogoods.goods_specification,  
ogoods.goods_type,  
ogoods.goods_brokerage,  
ogoods.is_refund as is_goods_refund,  
SUBSTRING(o.create_date,1,10) as dt  
  
--订单表  
from yp_dwd.fact_shop_order o  
  
--订单副表  
left join yp_dwd.fact_shop_order_address_detail od on o.id = od.id and od.end_date='9999-99-99'  
  
--订单组表  
left join yp_dwd.fact_shop_order_group og on o.id = og.order_id and og.end_date='9999-99-99'  
  
--订单组支付表  
left join yp_dwd.fact_order_pay op on og.group_id = op.group_id  
  
--退款单表  
left join yp_dwd.fact_refund_order refund on refund.order_id=o.id and  
refund.end_date='9999-99-99'  
  
--结算单表
```




```
left join yp_dwd.fact_order_settle os on os.order_id=o.id and os.end_date='9999-99-99'

--订单商品快照

left join yp_dwd.fact_shop_order_goods_details ogoods on ogoods.order_id=o.id and
ogoods.end_date='9999-99-99'

--订单评价表

left join yp_dwd.fact_goods_evaluation e on e.order_id=o.id and e.is_valid=1

--订单配送表

left join yp_dwd.fact_order_delievery_item d on d.shop_order_id=o.id and
d.dispatcher_order_type=1 and d.is_valid=1 and d.end_date='9999-99-99'

where o.end_date='9999-99-99';
```

2.3.4.3 执行计划对比

左侧开启mapjoin，右侧关闭mapjoin：

The image displays two side-by-side Hive execution plans. The left plan, with 'mapjoin' enabled, features a 'Map Join Operator' in the condition map, which lists seven 'Left Outer Join' operations. The right plan, with 'mapjoin' disabled, features a standard 'Join Operator' in the condition map, also listing seven 'Left Outer Join' operations. Both plans show a 'Filter Operator' at the top and a 'File Output Operator' at the bottom. The statistics for both plans are identical: 'Num rows: 15808 Data size: 28489176 Basic stats: COMPLETE Column stats: COMPLETE'.

2.3.4.4 实际执行时间对比

左侧开启mapjoin，右侧关闭mapjoin：

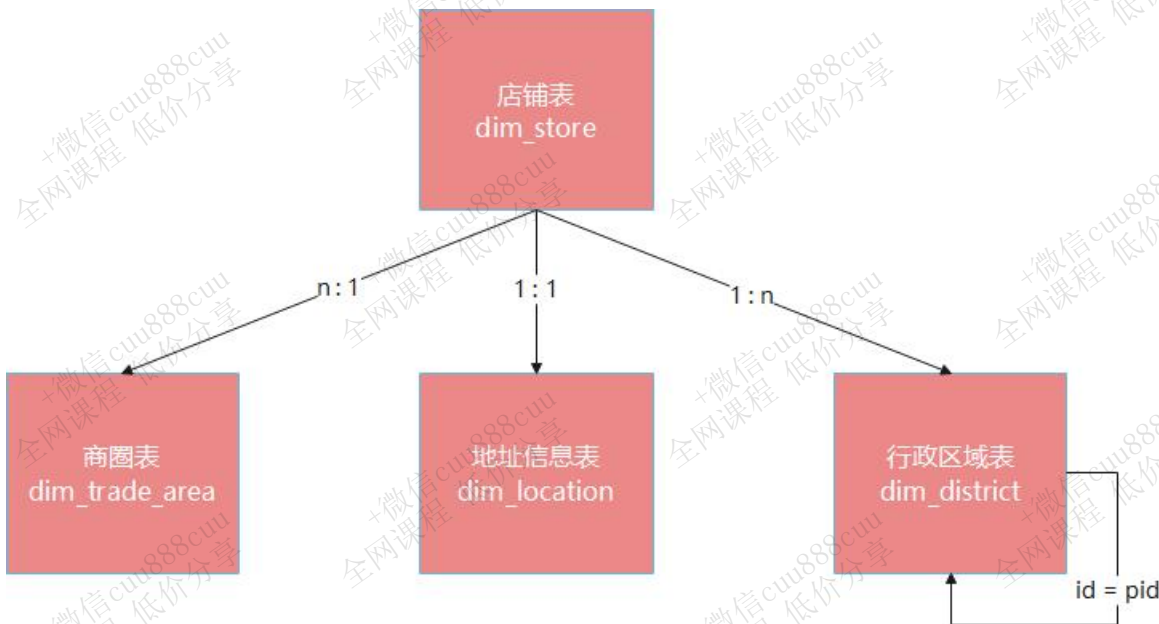
The image displays two screenshots of Hive execution logs. The left screenshot shows the execution time for a query with 'mapjoin' enabled, with a total MapReduce CPU time of 9 seconds 820 msec. The right screenshot shows the execution time for a query with 'mapjoin' disabled, with a total MapReduce CPU time of 1 minutes 29 seconds 360 msec. Both logs show the same query ID and command.





3. 店铺明细宽表

在DWB层，涉及到自关联的数据，要把相关的上下级数据都查询出来进行冗余，便于后续统计使用。



3.1 建表

```
DROP TABLE IF EXISTS yp_dwb.dwb_shop_detail;
CREATE TABLE yp_dwb.dwb_shop_detail (
-- 店铺
    id string,
    address_info string COMMENT '店铺详细地址',
    store_name string COMMENT '店铺名称',
    is_pay_bond tinyint COMMENT '是否有交过保证金 1: 是 0: 否',
    trade_area_id string COMMENT '归属商圈ID',
    delivery_method tinyint COMMENT '配送方式 1: 自提; 3: 自提加配送均可; 2: 商家配送',
    store_type int COMMENT '店铺类型 22天街网店 23实体店 24直营店铺 33会员专区店',
    is_primary tinyint COMMENT '是否是总店 1: 是 2: 不是',
    parent_store_id string COMMENT '父级店铺的id, 只有当is_primary类型为2时有效',
-- 商圈
    trade_area_name string COMMENT '商圈名称',
-- 区域-店铺
    province_id string COMMENT '店铺所在省份ID',
```





```
city_id string COMMENT '店铺所在城市ID',
area_id string COMMENT '店铺所在县ID',
province_name string COMMENT '省份名称',
city_name string COMMENT '城市名称',
area_name string COMMENT '县名称'
)
COMMENT '店铺明细表'
PARTITIONED BY (dt STRING)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

3.2 导入数据

3.2.1 首次

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
--分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;
--开启mapjoin
set hive.auto.convert.join=true;
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;
set hive.optimize.bucketmapjoin = true;
set hive.optimize.bucketmapjoin.sortedmerge=true;
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;

insert into yp_dwb.dwb_shop_detail partition (dt)
select
    s.id,
    s.address_info,
```





```
s.name as store_name,
s.is_pay_bond,
s.trade_area_id,
s.delivery_method,
s.store_type,
s.is_primary,
s.parent_store_id,
ta.name as trade_area_name,
pro.id province_id,
city.id city_id,
area.id area_id,
pro.name province_name,
city.name city_name,
area.name area_name,
SUBSTRING(s.create_time,1,10) dt
--店铺
from yp_dwd.dim_store s
--商圈
left join yp_dwd.dim_trade_area ta on s.trade_area_id=ta.id and ta.end_date='9999-99-99'
-- 地区
left join yp_dwd.dim_location lo on lo.type=2 and lo.correlation_id=s.id and
lo.end_date='9999-99-99'
left join yp_dwd.dim_district_area on area.code=lo.adcode
left join yp_dwd.dim_district_city on area.pid=city.id
left join yp_dwd.dim_district_pro on city.pid=pro.id
where s.end_date='9999-99-99'
;
```

3.2.2 循环

店铺入驻有审核流程，一旦审核通过后就不能再修改。

确定业务更新周期后，只需要将更新周期内的数据覆盖插入即可。通过 `create_time` 限制数据范围，通过 `start_date` 走分区提高性能。

```
--增量覆盖插入
insert overwrite table yp_dwb.dwb_shop_detail partition (dt)
select
.....,
SUBSTRING(s.create_time,1,10) dt
--店铺
from yp_dwd.dim_store s
```





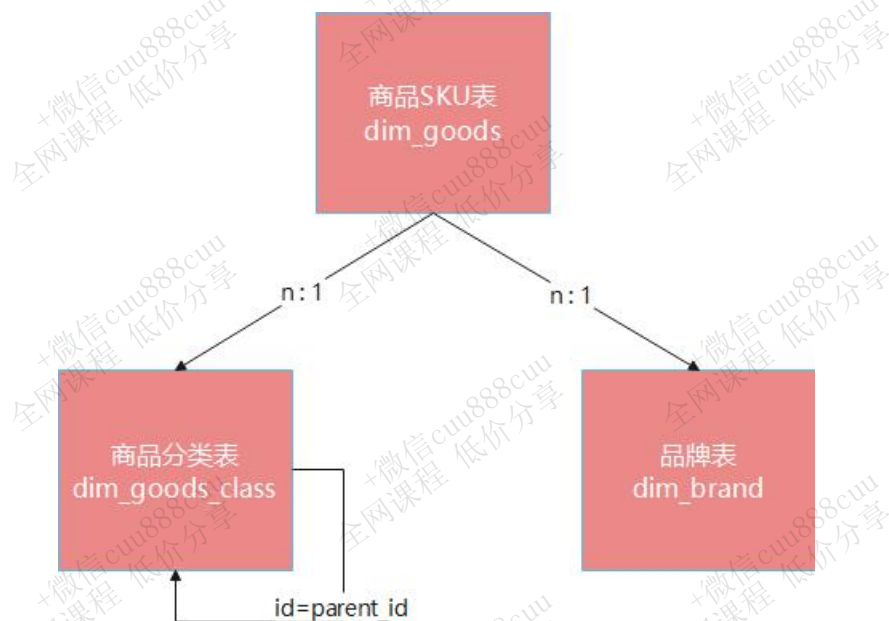
```
--商圈
left join yp_dwd.dim_trade_area ta on s.trade_area_id=ta.id and ta.end_date='9999-99-99'

-- 地区
left join yp_dwd.dim_location lo on lo.type=2 and lo.correlation_id=s.id and
lo.end_date='9999-99-99'

left join yp_dwd.dim_district_area on area.code=lo.adcode
left join yp_dwd.dim_district_city on area.pid=city.id
left join yp_dwd.dim_district_pro on city.pid=pro.id
where s.end_date='9999-99-99'

-- 读取上个月1日至今的数据
and SUBSTRING(s.create_time,1,10) >= '${Last_Month_DATE}' and s.start_date >=
'${Last_Month_DATE}';
```

4. 商品明细宽表



4.1 建表

```
DROP TABLE if EXISTS yp_dwb.dwb_goods_detail;
CREATE TABLE yp_dwb.dwb_goods_detail(
  id string,
  store_id string COMMENT '所属商店ID',
  class_id string COMMENT '分类id:只保存最后一层分类id',
  store_class_id string COMMENT '店铺分类id',
  brand_id string COMMENT '品牌id',
```





```
goods_name string COMMENT '商品名称',
goods_specification string COMMENT '商品规格',
search_name string COMMENT '模糊搜索名称字段:名称_+真实名称',
goods_sort int COMMENT '商品排序',
goods_market_price decimal(36,2) COMMENT '商品市场价',
goods_price decimal(36,2) COMMENT '商品销售价格(原价)',
goods_promotion_price decimal(36,2) COMMENT '商品促销价格(售价)',
goods_storage int COMMENT '商品库存',
goods_limit_num int COMMENT '购买限制数量',
goods_unit string COMMENT '计量单位',
goods_state tinyint COMMENT '商品状态 1正常, 2下架,3违规(禁售)',
goods_verify tinyint COMMENT '商品审核状态: 1通过, 2未通过, 3审核中',
activity_type tinyint COMMENT '活动类型:0无活动1促销2秒杀3折扣',
discount int COMMENT '商品折扣(%)',
seckill_begin_time string COMMENT '秒杀开始时间',
seckill_end_time string COMMENT '秒杀结束时间',
seckill_total_pay_num int COMMENT '已秒杀数量',
seckill_total_num int COMMENT '秒杀总数限制',
seckill_price decimal(36,2) COMMENT '秒杀价格',
top_it tinyint COMMENT '商品置顶: 1-是, 0-否',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '0: 失效, 1: 开启',
-- 商品小类
min_class_id string COMMENT '分类id:只保存最后一层分类id',
min_class_name string COMMENT '店铺内分类名字',
-- 商品中类
mid_class_id string COMMENT '分类id:只保存最后一层分类id',
mid_class_name string COMMENT '店铺内分类名字',
-- 商品大类
max_class_id string COMMENT '分类id:只保存最后一层分类id',
max_class_name string COMMENT '店铺内分类名字',
-- 品牌
brand_name string COMMENT '品牌名称'
)
COMMENT '商品明细表'
PARTITIONED BY(dt STRING)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

4.2 导入数据

4.2.1 首次





```
INSERT into yp_dwb.dwb_goods_detail partition (dt)
```

```
SELECT
```

```
    goods.id,  
    goods.store_id,  
    goods.class_id,  
    goods.store_class_id,  
    goods.brand_id,  
    goods.goods_name,  
    goods.goods_specification,  
    goods.search_name,  
    goods.goods_sort,  
    goods.goods_market_price,  
    goods.goods_price,  
    goods.goods_promotion_price,  
    goods.goods_storage,  
    goods.goods_limit_num,  
    goods.goods_unit,  
    goods.goods_state,  
    goods.goods_verify,  
    goods.activity_type,  
    goods.discount,  
    goods.seckill_begin_time,  
    goods.seckill_end_time,  
    goods.seckill_total_pay_num,  
    goods.seckill_total_num,  
    goods.seckill_price,  
    goods.top_it,  
    goods.create_user,  
    goods.create_time,  
    goods.update_user,  
    goods.update_time,  
    goods.is_valid,
```

```
    CASE class1.level WHEN 3
```

```
        THEN class1.id
```

```
        ELSE NULL
```

```
    END as min_class_id,
```

```
    CASE class1.level WHEN 3
```

```
        THEN class1.name
```

```
        ELSE NULL
```

```
    END as min_class_name,
```

```
    CASE WHEN class1.level=2
```

```
        THEN class1.id
```





```
        WHEN class2.level=2
        THEN class2.id
        ELSE NULL
    END as mid_class_id,
    CASE WHEN class1.level=2
        THEN class1.name
        WHEN class2.level=2
        THEN class2.name
        ELSE NULL
    END as mid_class_name,
    CASE WHEN class1.level=1
        THEN class1.id
        WHEN class2.level=1
        THEN class2.id
        WHEN class3.level=1
        THEN class3.id
        ELSE NULL
    END as max_class_id,
    CASE WHEN class1.level=1
        THEN class1.name
        WHEN class2.level=1
        THEN class2.name
        WHEN class3.level=1
        THEN class3.name
        ELSE NULL
    END as max_class_name,
    brand.brand_name,
    SUBSTRING(goods.create_time,1,10) dt
--SKU
FROM yp_dwd.dim_goods goods
--商品分类
left join yp_dwd.dim_goods_class class1 on goods.store_class_id = class1.id AND
class1.end_date='9999-99-99'
left join yp_dwd.dim_goods_class class2 on class1.parent_id = class2.id AND
class2.end_date='9999-99-99'
left join yp_dwd.dim_goods_class class3 on class2.parent_id = class3.id AND
class3.end_date='9999-99-99'
--品牌
left join yp_dwd.dim_brand brand on goods.brand_id=brand.id AND brand.end_date='9999-99-99'
WHERE goods.end_date='9999-99-99'
;
```



4.2.2 循环

商品发布有审核流程，一旦审核通过后就不能再修改；如要修改需重新发布。

确定业务更新周期后，只需要将更新周期内的数据覆盖插入即可。通过 `create_time` 限制数据范围，通过 `start_date` 走分区提高性能。

```
--先删除，测试用
ALTER TABLE yp_dwb.dwb_goods_detail DROP IF EXISTS PARTITION (dt>='${Last_Month_DATE}');

--增量插入
INSERT into yp_dwb.dwb_goods_detail partition (dt)
SELECT
    ...../
    brand.brand_name,
    SUBSTRING(goods.create_time,1,10) dt
FROM yp_dwd.dim_goods goods

--商品分类
left join yp_dwd.dim_goods_class class1 on goods.store_class_id = class1.id AND
class1.end_date='9999-99-99'

left join yp_dwd.dim_goods_class class2 on class1.parent_id = class2.id AND
class2.end_date='9999-99-99'

left join yp_dwd.dim_goods_class class3 on class2.parent_id = class3.id AND
class3.end_date='9999-99-99'

--品牌
left join yp_dwd.dim_brand brand on goods.brand_id=brand.id AND brand.end_date='9999-99-99'
WHERE goods.end_date='9999-99-99'

-- 读取上个月1日至今的数据
and SUBSTRING(goods.create_time,1,10) >= '${Last_Month_DATE}' and goods.start_date >=
 '${Last_Month_DATE}';
```

4.2.3 Hive索引

Hive支持索引，但是Hive的索引与关系型数据库中的索引并不相同，比如，Hive不支持主键或者外键。

Hive索引可以建立在表中的某些列上，以提升一些操作的效率，例如减少MapReduce任务中需要读取的数据块的数量。

在可以预见到分区数据非常庞大的情况下，分桶和索引常常是优于分区的。而分桶由于SMB Join对关联键要求严格，所以并不是总能生效。



4.2.3.1 Hive原始索引

Hive的索引目的是提高Hive表指定列的查询速度。

没有索引时，类似'WHERE tab1.col1 = 10' 的查询，Hive会加载整张表或分区，然后处理所有的rows，但是如果在字段col1上面存在索引时，那么只会加载和处理文件的一部分。

在每次建立、更新数据后，Hive索引不会自动更新，需要手动进行更新（**重建索引**以构建索引表），会触发一个mr job。

Hive索引使用过程繁杂，而且性能一般，在Hive3.0中已被删除，在工作环境中不推荐优先使用，在分区数量过多或查询字段不是分区字段时，索引可以作为补充方案同时使用。推荐使用**ORC文件格式的索引类型**进行查询。

4.2.3.2 Row Group Index

一个ORC文件包含一个或多个stripes(groups of row data)，每个stripe中包含了每个column的**min/max值**的索引数据，当查询中有<, >, =的操作时，会根据min/max值，跳过扫描不包含的stripes。

而其中为每个stripe建立的包含min/max值的索引，就称为**Row Group Index行组索引**，也叫**min-max Index大小对比索引**，或者Storage Index。

在建立ORC格式表时，指定表参数'**orc.create.index='true'**'之后，便会建立Row Group Index，需要注意的是，为了使Row Group Index有效利用，**向表中加载数据时，必须对需要使用索引的字段进行排序**，否则，min/max会失去意义。另外，这种索引主要用于**数值型字段**的查询过滤优化上。

设置**hive.optimize.index.filter**为true，并重启hive



创建表/插入数据：

```
CREATE TABLE lxw1234_orc2 stored AS ORC
TBLPROPERTIES
(
  'orc.compress'='SNAPPY',
  -- 开启行组索引
  'orc.create.index'='true'
)
AS
SELECT CAST(siteid AS INT) AS id,
pcid
FROM lxw1234_text
-- 插入的数据保持排序
DISTRIBUTE BY id sort BY id;
```

查询：

```
set hive.optimize.index.filter=true;
SELECT COUNT(1) FROM lxw1234_orc1 WHERE id >= 1382 AND id <= 1399;
```

4.2.3.3 Bloom Filter Index

在建表时候，通过表参数 `"orc.bloom.filter.columns"="pcid"` 来指定为那些字段建立 BloomFilter 索引，这样，在生成数据的时候，会在每个 stripe 中，为该字段建立 BloomFilter 的数据结构，当查询条件中包含对该字段的 **=号过滤** 时候，先从 BloomFilter 中获取以下是否包含该值，如果不包含，则跳过该 stripe。

创建：

```
CREATE TABLE lxw1234_orc2 stored AS ORC
TBLPROPERTIES
(
  'orc.compress'='SNAPPY',
  'orc.create.index'='true',
  -- pcid 字段开启 BloomFilter 索引
  "orc.bloom.filter.columns"="pcid"
)
AS
SELECT CAST(siteid AS INT) AS id,
pcid
FROM lxw1234_text
DISTRIBUTE BY id sort BY id;
```

查询：

```
SET hive.optimize.index.filter=true;
SELECT COUNT(1) FROM lxw1234_orc1 WHERE id >= 0 AND id <= 1000
AND pcid IN ('0005E26F0DCCDB56F9041C', 'A');
```

只有在数据量较大时，使用索引才能带来性能优势。





4.2.4 索引方式实现

4.2.4.1 重建为索引表

将来源的DWD层表重建为索引表后，重新插入数据：

```
DROP TABLE if EXISTS yp_dwd.dim_goods;
CREATE TABLE yp_dwd.dim_goods (
  id string,
  store_id string COMMENT '所属商店ID',
  class_id string COMMENT '分类id:只保存最后一层分类id',
  store_class_id string COMMENT '店铺分类id',
  brand_id string COMMENT '品牌id',
  goods_name string COMMENT '商品名称',
  goods_specification string COMMENT '商品规格',
  search_name string COMMENT '模糊搜索名称字段:名称_+真实名称',
  goods_sort int COMMENT '商品排序',
  goods_market_price decimal(36,2) COMMENT '商品市场价',
  goods_price decimal(36,2) COMMENT '商品销售价格(原价)',
  goods_promotion_price decimal(36,2) COMMENT '商品促销价格(售价)',
  goods_storage int COMMENT '商品库存',
  goods_limit_num int COMMENT '购买限制数量',
  goods_unit string COMMENT '计量单位',
  goods_state tinyint COMMENT '商品状态 1正常, 2下架, 3违规(禁售)',
  goods_verify tinyint COMMENT '商品审核状态: 1通过, 2未通过, 3审核中',
  activity_type tinyint COMMENT '活动类型: 0无活动 1促销 2秒杀 3折扣',
  discount int COMMENT '商品折扣(%)',
  seckill_begin_time string COMMENT '秒杀开始时间',
  seckill_end_time string COMMENT '秒杀结束时间',
  seckill_total_pay_num int COMMENT '已秒杀数量',
  seckill_total_num int COMMENT '秒杀总数限制',
  seckill_price decimal(36,2) COMMENT '秒杀价格',
  top_it tinyint COMMENT '商品置顶: 1-是, 0-否',
  create_user string,
  create_time string,
  update_user string,
```





```
update_time string,  
is_valid tinyint COMMENT '0 : 失效, 1 : 开启',  
end_date string COMMENT '拉链结束日期')  
COMMENT '商品表_店铺(SKU)'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY', 'orc.create.index'='true',  
'orc.bloom.filter.columns'='store_class_id,end_date');  
  
DROP TABLE if EXISTS yp_dwd.dim_goods_class;  
CREATE TABLE yp_dwd.dim_goods_class(  
id string,  
store_id string COMMENT '店铺id',  
class_id string COMMENT '对应的平台分类表id',  
name string COMMENT '店铺内分类名字',  
parent_id string COMMENT '父id',  
level tinyint COMMENT '分类层级',  
is_parent_node tinyint COMMENT '是否为父节点:1是0否',  
background_img string COMMENT '背景图片',  
img string COMMENT '分类图片',  
keywords string COMMENT '关键词',  
title string COMMENT '搜索标题',  
sort int COMMENT '排序',  
note string COMMENT '类型描述',  
url string COMMENT '分类的链接',  
is_use tinyint COMMENT '是否使用:0否,1是',  
create_user string,  
create_time string,  
update_user string,  
update_time string,  
is_valid tinyint COMMENT '0 : 失效, 1 : 开启',  
end_date string COMMENT '拉链结束日期')  
COMMENT '商品分类表'  
partitioned by (start_date string)
```





```
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY', 'orc.create.index'='true',
'orc.bloom.filter.columns'='id,parent_id,end_date');

DROP TABLE if EXISTS yp_dwd.dim_brand;
CREATE TABLE yp_dwd.dim_brand(
    id string,
    store_id string COMMENT '店铺id',
    brand_pt_id string COMMENT '平台品牌库品牌Id',
    brand_name string COMMENT '品牌名称',
    brand_image string COMMENT '品牌图片',
    initial string COMMENT '品牌首字母',
    sort int COMMENT '排序',
    is_use tinyint COMMENT '0禁用1启用',
    goods_state tinyint COMMENT '商品品牌审核状态 1 审核中,2 通过,3 拒绝',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid tinyint COMMENT '0 : 失效, 1 : 开启',
    end_date string COMMENT '拉链结束日期')
COMMENT '品牌 (店铺)'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY', 'orc.create.index'='true',
'orc.bloom.filter.columns'='id,end_date');

--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
```





```
--hive压缩

set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;

--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;


--全量
INSERT overwrite TABLE yp_dwd.dim_goods PARTITION(start_date)
SELECT
  id,
  store_id,
  class_id,
  store_class_id,
  brand_id,
  goods_name,
  goods_specification,
  search_name,
  goods_sort,
  goods_market_price,
  goods_price,
  goods_promotion_price,
  goods_storage,
  goods_limit_num,
  goods_unit,
  goods_state,
  goods_verify,
  activity_type,
  discount,
  seckill_begin_time,
  seckill_end_time,
  seckill_total_pay_num,
  seckill_total_num,
  seckill_price,
  top_it,
  create_user,
  create_time,
  update_user,
  update_time,
  is_valid,
  '9999-99-99' end_date,
  dt as start_date
```





```
FROM yp_ods.t_goods;
```

```
--全量
```

```
INSERT overwrite TABLE yp_dwd.dim_goods_class PARTITION(start_date)
```

```
SELECT
```

```
    id,  
    store_id,  
    class_id,  
    name,  
    parent_id,  
    level,  
    is_parent_node,  
    background_img,  
    img,  
    keywords,  
    title,  
    sort,  
    note,  
    url,  
    is_use,  
    create_user,  
    create_time,  
    update_user,  
    update_time,  
    is_valid,  
    '9999-99-99' end_date,  
    dt as start_date
```

```
FROM yp_ods.t_goods_class;
```

```
--全量
```

```
INSERT overwrite TABLE yp_dwd.dim_brand PARTITION(start_date)
```

```
SELECT
```

```
    id,  
    store_id,  
    brand_pt_id,  
    brand_name,  
    brand_image,  
    initial,  
    sort,  
    is_use,
```





```
goods_state,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
dt as start_date  
FROM yp_ods.t_brand;
```

4.2.4.2 明细宽表导入数据

同4.2.1，不变。

```
INSERT into yp_dwb.dwb_goods_detail partition (dt)  
SELECT  
    goods.id,  
    .....  
--SKU  
FROM yp_dwd.dim_goods goods  
--商品分类  
left join yp_dwd.dim_goods_class class1 on goods.store_class_id = class1.id AND  
class1.end_date='9999-99-99'  
left join yp_dwd.dim_goods_class class2 on class1.parent_id = class2.id AND  
class2.end_date='9999-99-99'  
left join yp_dwd.dim_goods_class class3 on class2.parent_id = class3.id AND  
class3.end_date='9999-99-99'  
--品牌  
left join yp_dwd.dim_brand brand on goods.brand_id=brand.id AND brand.end_date='9999-99-99'  
WHERE goods.end_date='9999-99-99'  
;
```

